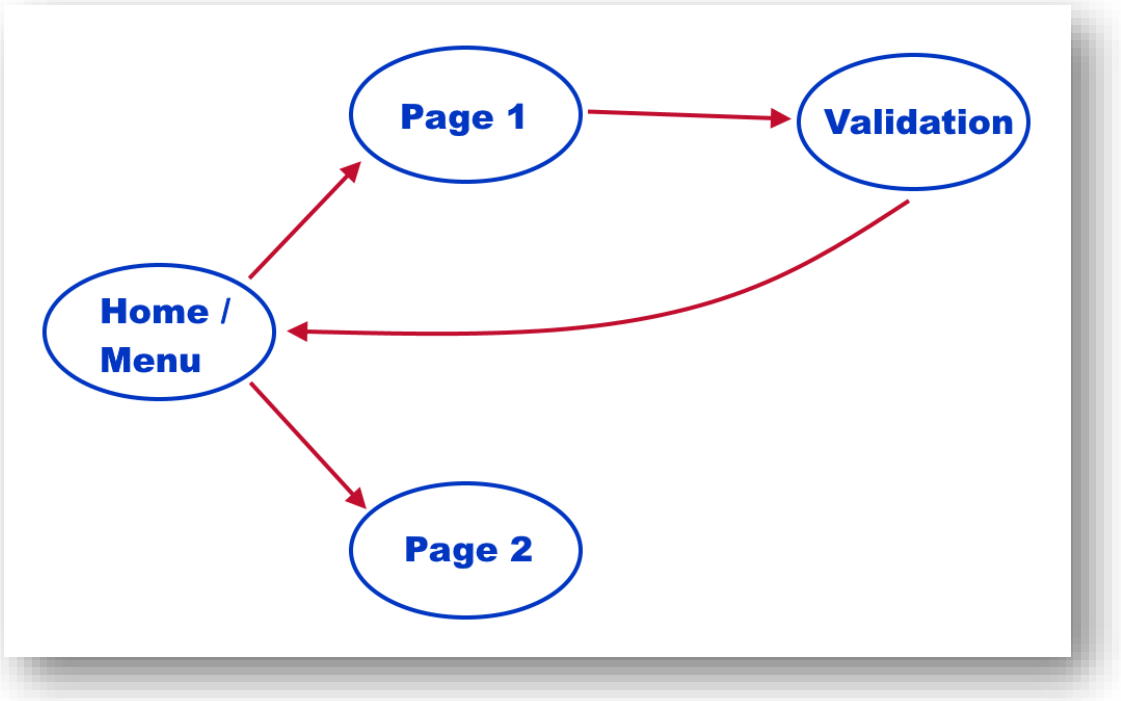
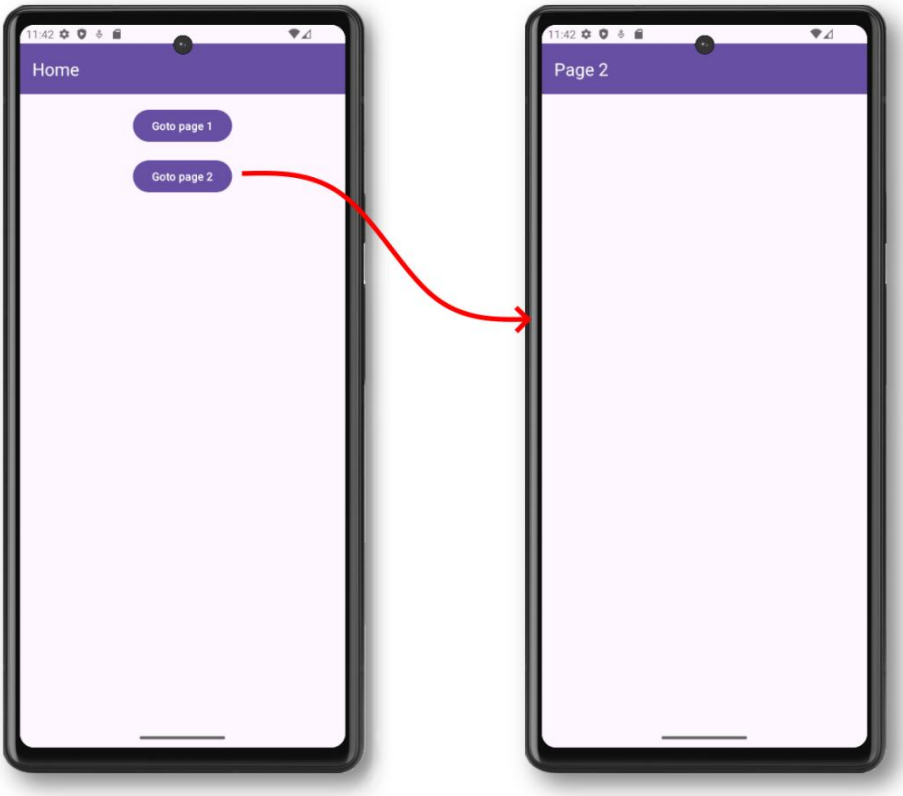
# Preamble : Fragments

Let us say we must create an application with the following structure:



Creating the skeleton of this app would involve creating 4 Activity classes and lots of Intent interaction.

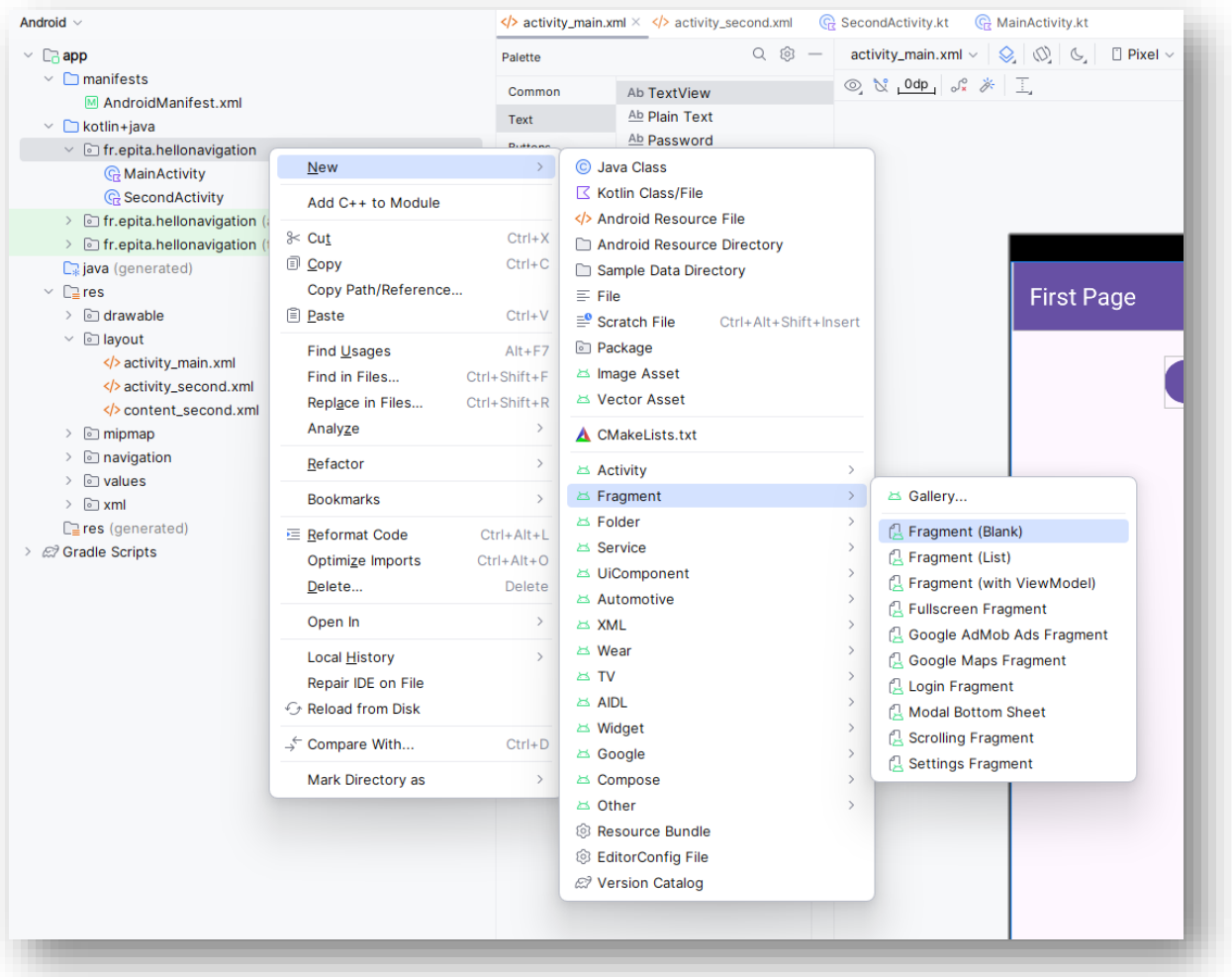
But…

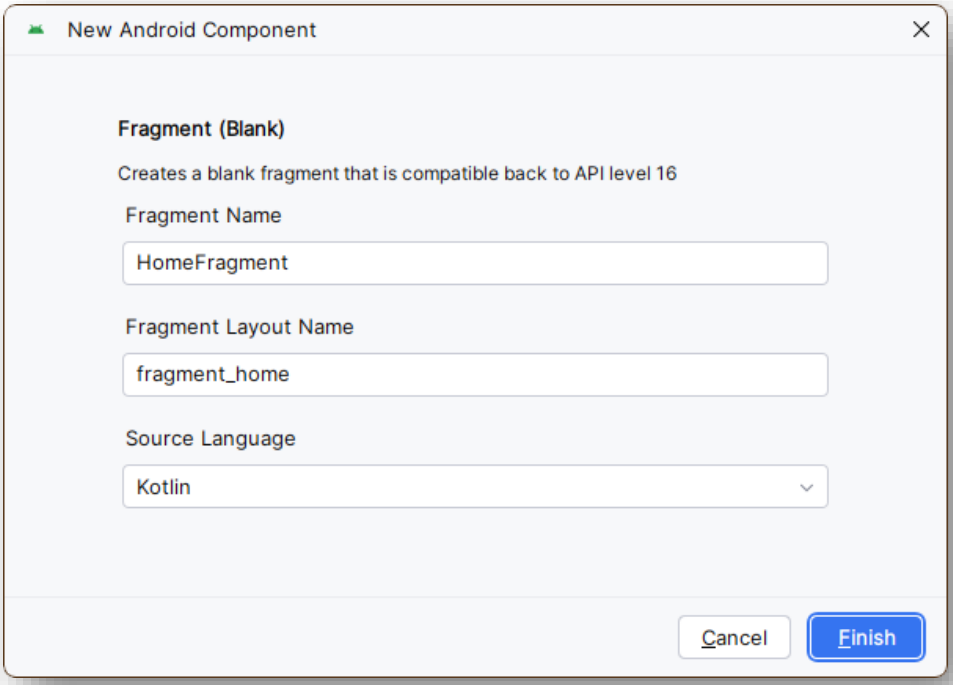
- Such activities look like god classes that directly conflict with the principle of separation of concerns.
- Activity isolation makes data sharing difficult.
- Activities are not very modular therefore not very re-usable or easy to combine.
- Navigating from one activity to another looks and feels like moving from one application to a different one.

A first step towards better app architecture would be to extract the “display responsibility”. To achieve this, the Activity will remain as some kind of backbone, directing and managing what to display and when, and graphical content, or pages, will be formalized in modular components called Fragments.

### *Step 1: Create the first fragment*

Default “empty” Fragments code depends on various parameters (Studio version, build tools, chosen template, etc.) therefore may contain a lot of useless boilerplate code but you can keep only core parts *onCreateView()* and *onViewCreated()* callback methods:






```kotlin
class HomeFragment : Fragment() {
    // onCreateView : called every time the fragment needs to draw its UI
    //                Must return the inflated fragment layout view
    // params :
    // inflater : a ready to use inflater. Use it to inflate fragment layout
    // container : the viewgroup that will hold the fragment
    // savedInstanceState : fragment saved state information if it has
    //                      already been saved, else null
    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        // Inflate the layout for this fragment
        val fragmentView: View = inflater.inflate(
            R.layout.fragment_home,
            container,
            false
        )

        return fragmentView
    }

    //onViewCreated : called immediately after onCreateView
    //                Initialize view behavior here
    // params :
    // view : fragment view, returned by onCreateView()
    // savedInstanceState : fragment saved state information if it has
    //                      already been saved, else null
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        val goto2Button: Button = view.findViewById(R.id.btn_goto2)
        goto2Button.setOnClickListener { /* do stuff....*/ }

    }
}
```

Fragments and Activities both rely on layout files to describe their UI but their content is quite different:

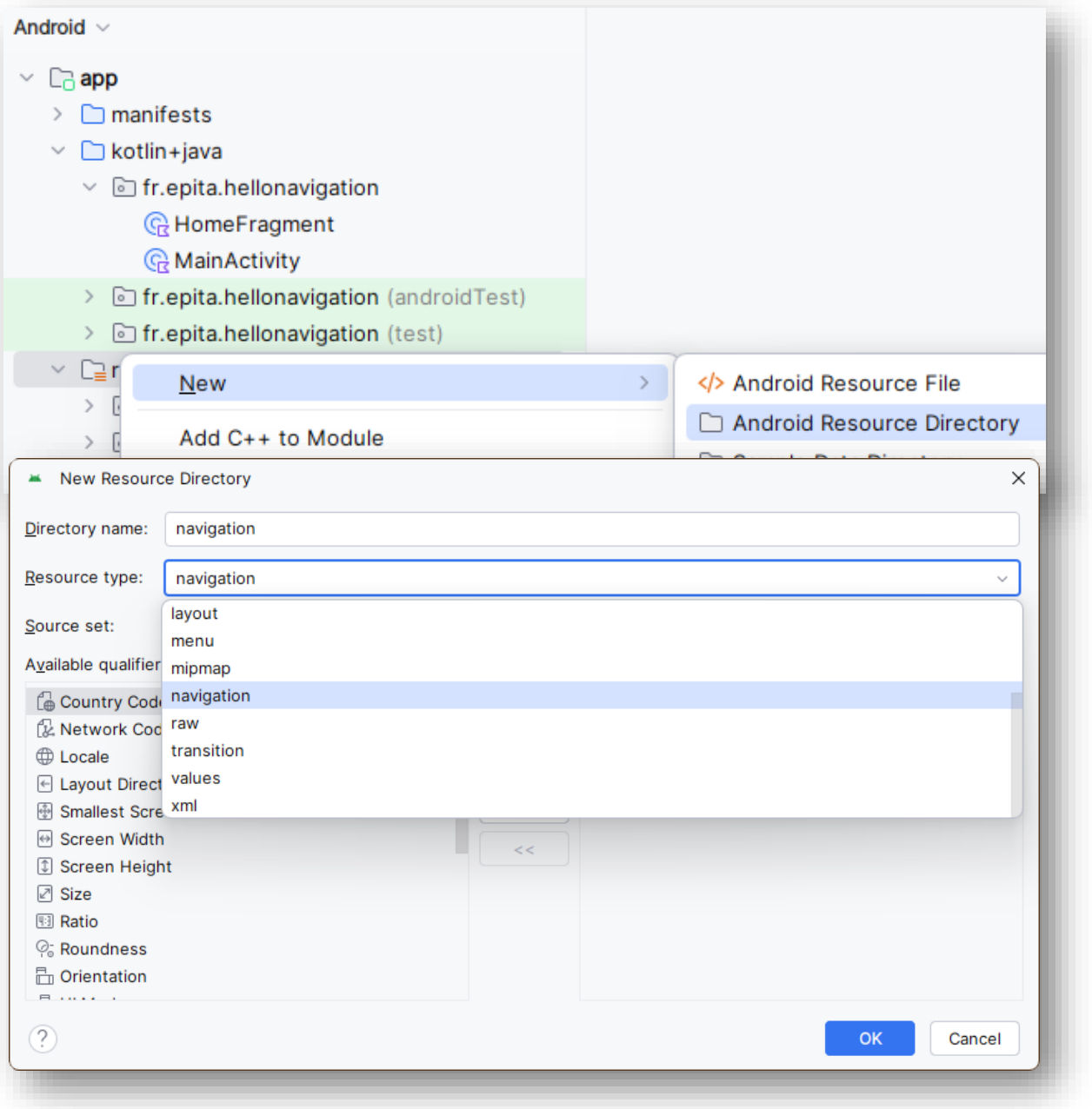
- Fragment layout files are visible screen content, which is why they are virtually identical to the ones previously used for activities.
- Activity layout files contain the structure that will display fragments. Additionally, they can contain graphical components that are identical in all screens (like the actionbar for example).

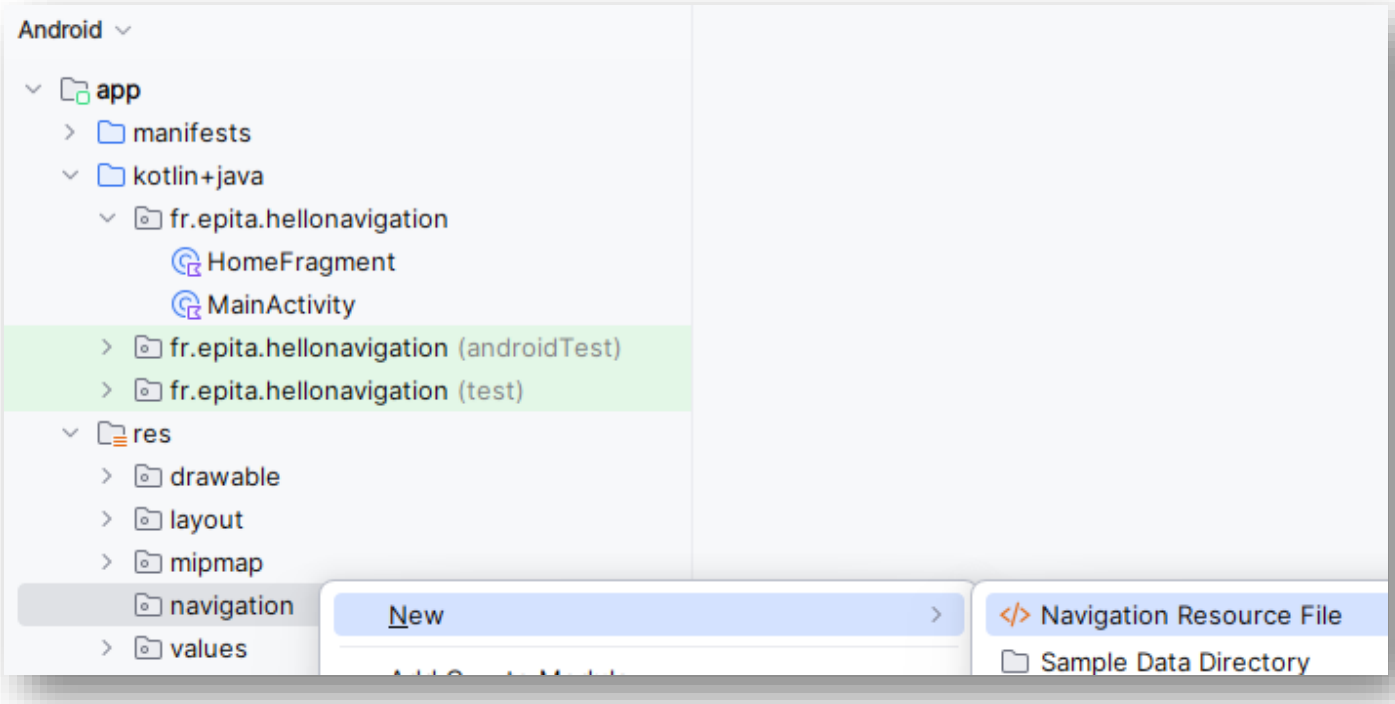
## Navigation component

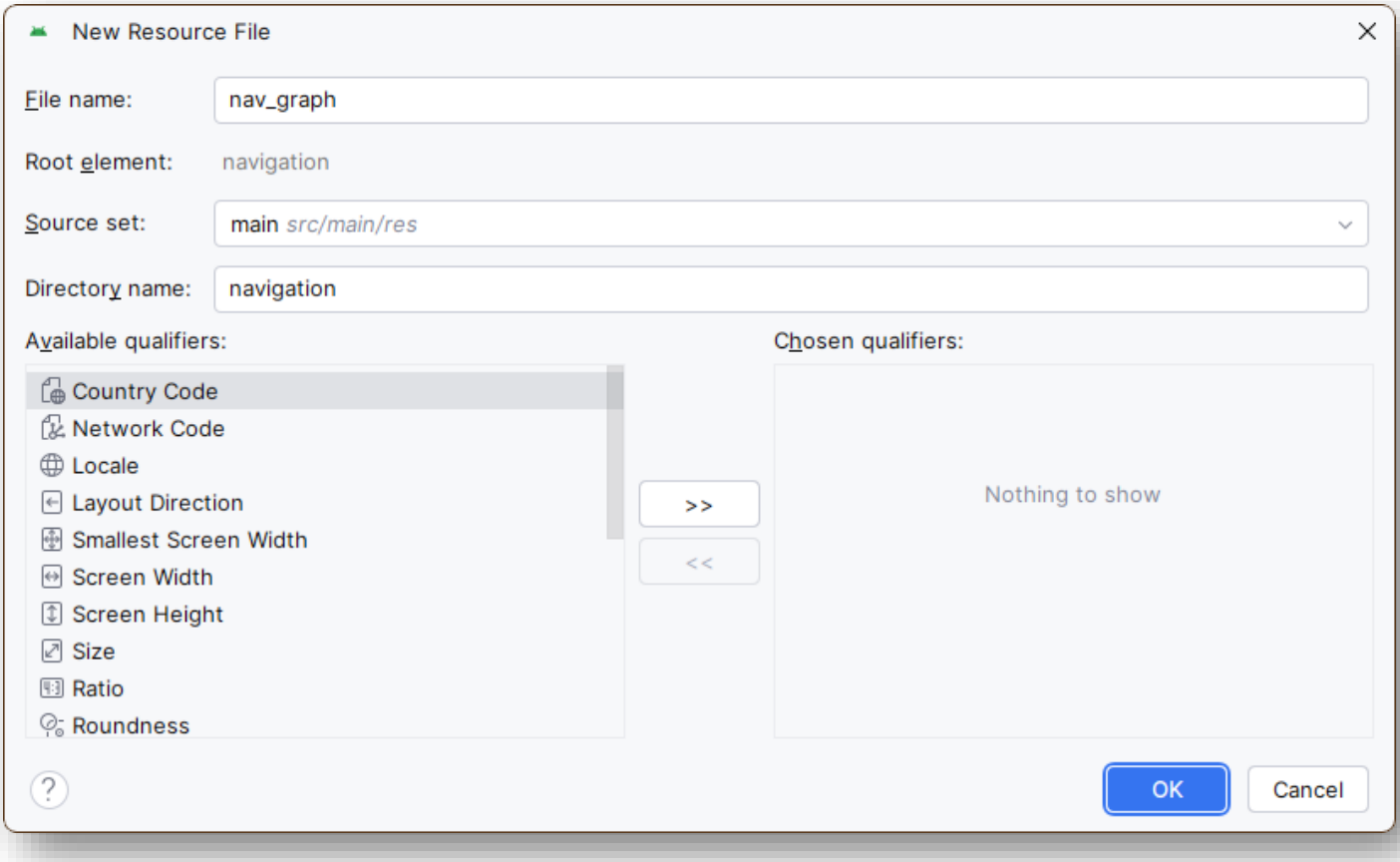
*Step 2: display home fragment*

The *fragment manager* API is used by the activity to handle fragments. Although it can be used directly, its use is abstracted through the *navigation component*.

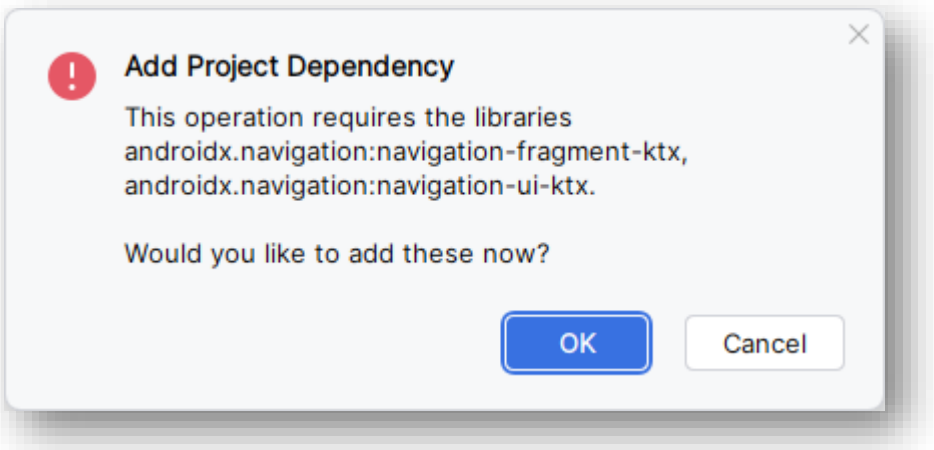
Navigation, or screens organization, is described in a graph, which is a resource that must be added to the project.




Android
app
manifests
kotlin+java
fr.epita.hellonavigation
HomeFragment
MainActivity
fr.epita.hellonavigation (androidTest)
fr.epita.hellonavigation (test)
res
drawable
layout
mipmap
navigation
values
New
Navigation Resource File
Sample Data Directory


New Resource File
File name:
nav_graph
Root element:
navigation
Source set:
main src/main/res
Directory name:
navigation
Available qualifiers:
Country Code
Network Code
Locale
Layout Direction
Smallest Screen Width
Screen Width
Screen Height
Size
Ratio
Roundness
>>
<<
Chosen qualifiers:
Nothing to show
OK
Cancel

Adding the nav_graph should add the corresponding dependencies to module build.graddle file.

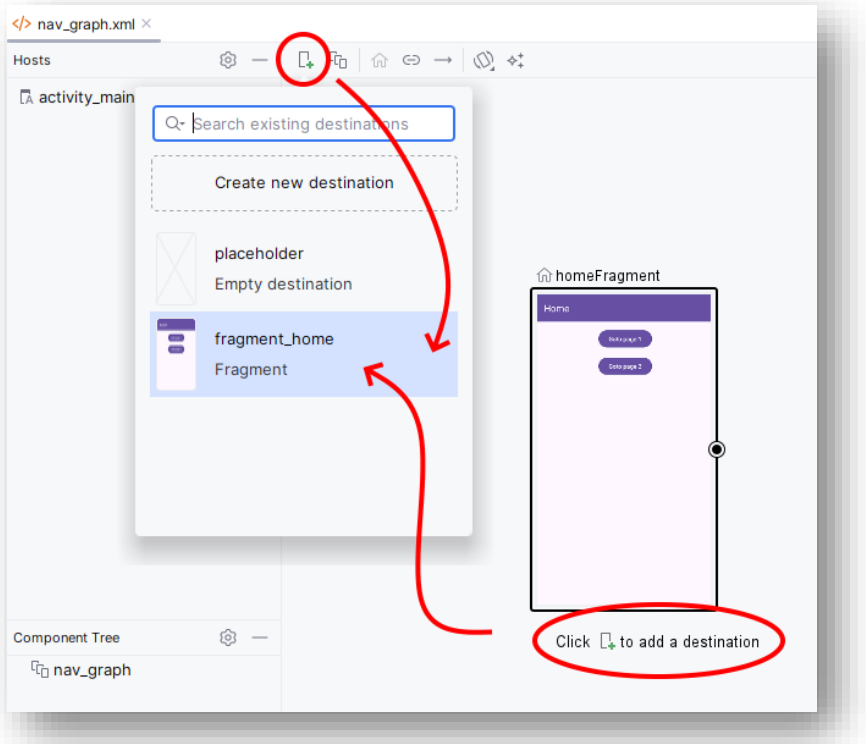



```
dependencies {
    val nav_version = "2.9.5"
    . . .

    // Add dependency on navigation component
    implementation("androidx.navigation:navigation-fragment-ktx: $nav_version")
    implementation("androidx.navigation:navigation-ui-ktx: $nav_version")
}
```

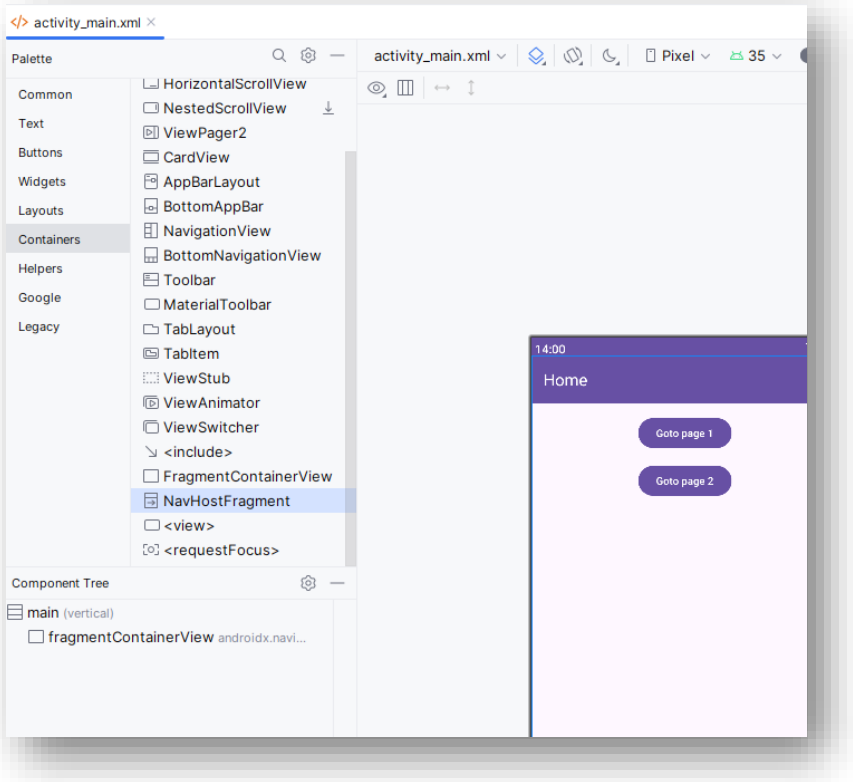
NB: dependencies are now split between build.gradle and lib.versions .toml file and do not appear as shown above. This presentation was chosen for convenience.

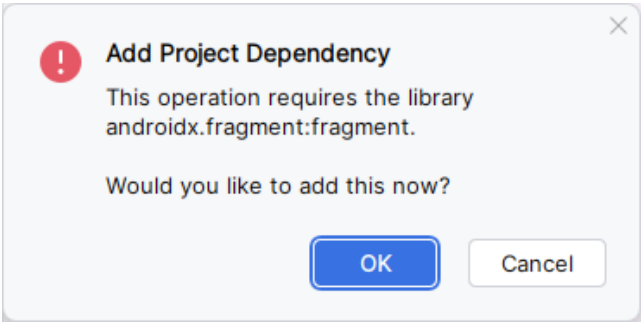
We will dive in the graph later but for now, just add the existing fragment:

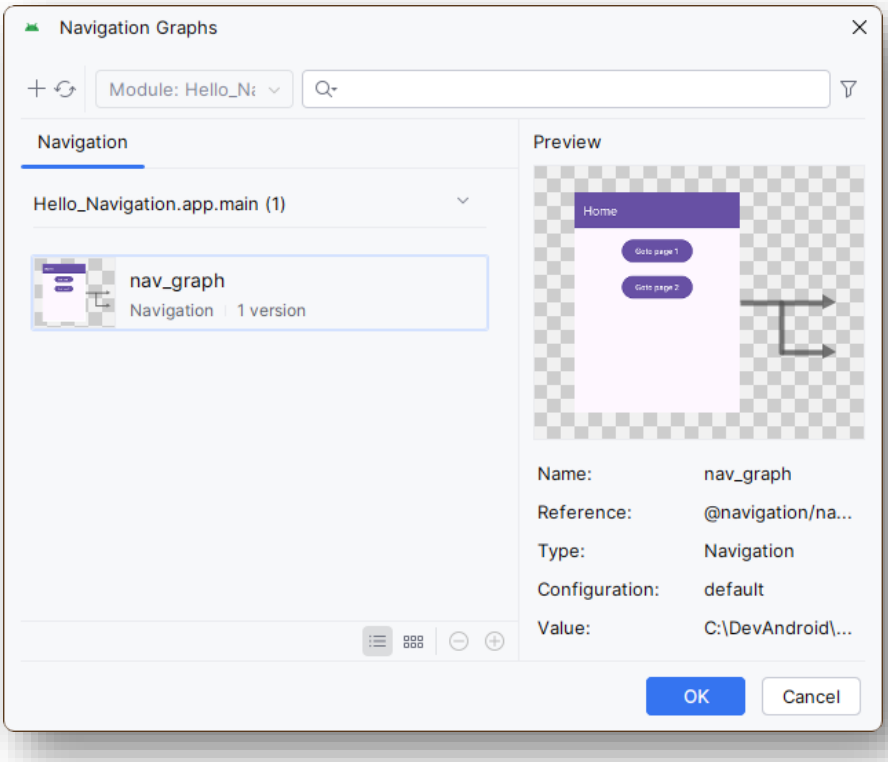


Now some space in Activity layout must be devoted to fragment display. Like a frame with dynamic content.

This area is the *NavHostFragment* container view. Add it to your activity layout and it should link to the existing nav_graph.



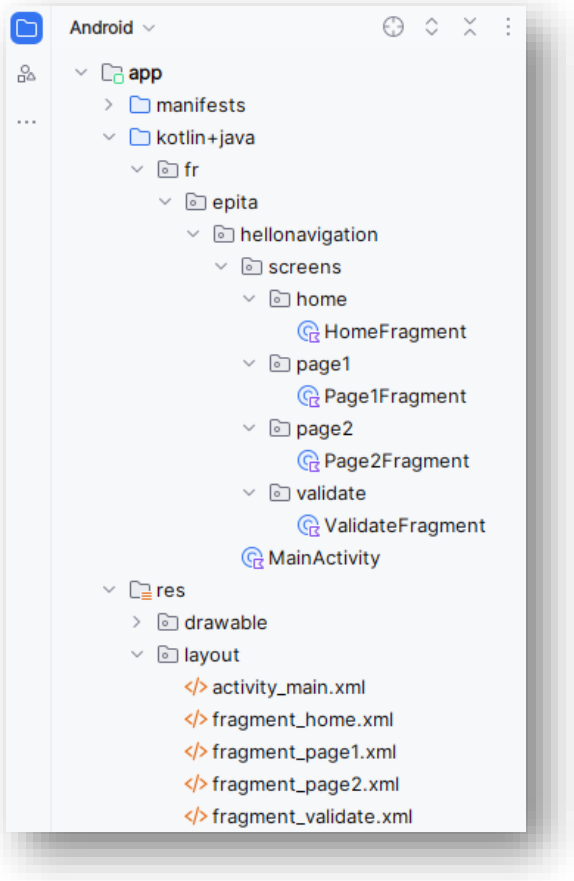

Et voila ! Run the app and it should show the first page.

*Step 3: add other fragments*

So far so good but with only one screen we did not see any real “navigation”!

But first thing first, let us add the other screens by creating the corresponding fragments and their layout files.

(fast forwarding those steps as there is nothing new here)



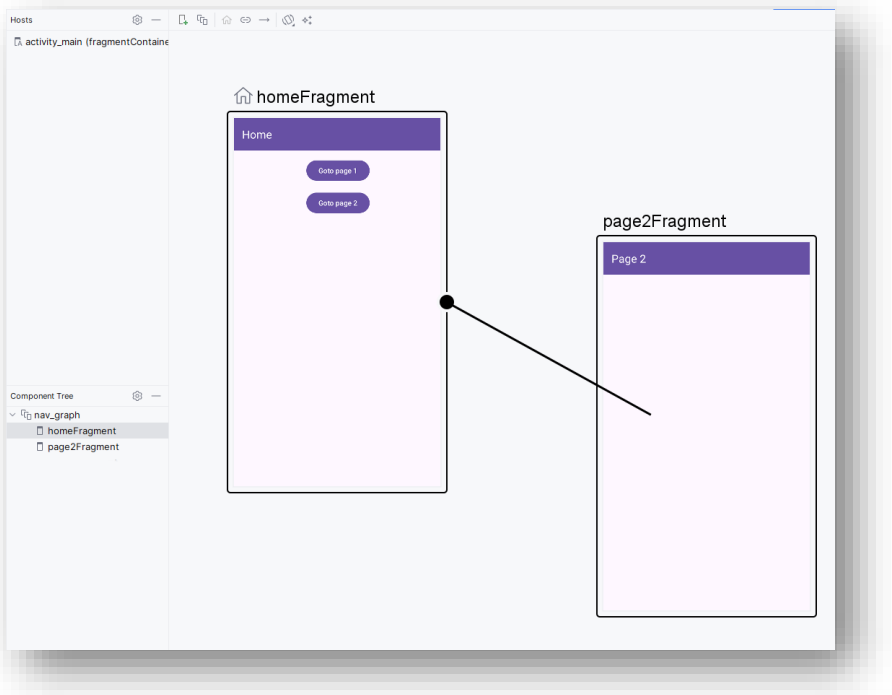

*Step 4: complete graph with fragments interaction*

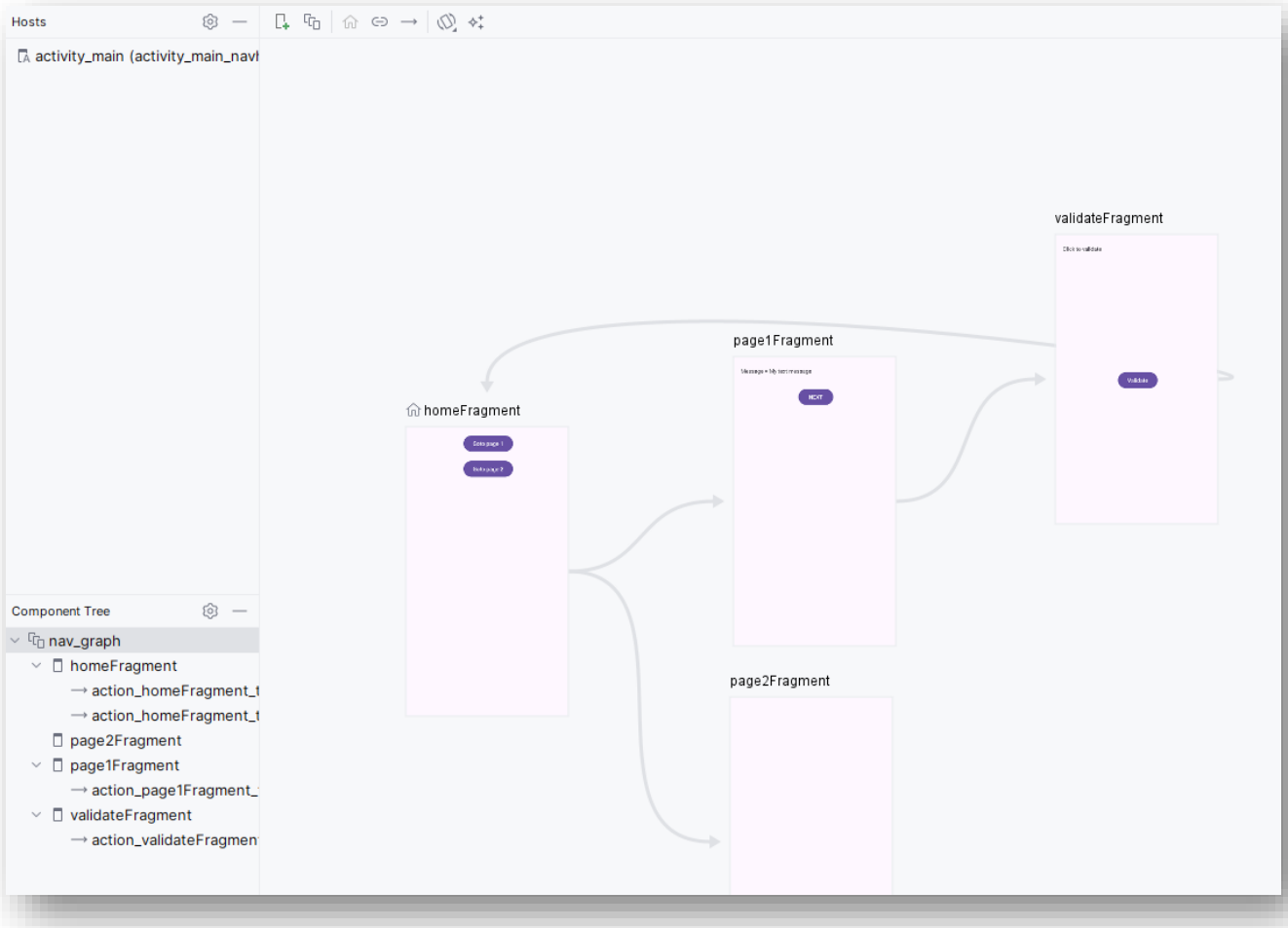
Back to the graph. It is a schema of all the screens in the app but also how they are related.

This is where you add your fragments as screens and create the relations (or ***actions*** as they are called) between them.

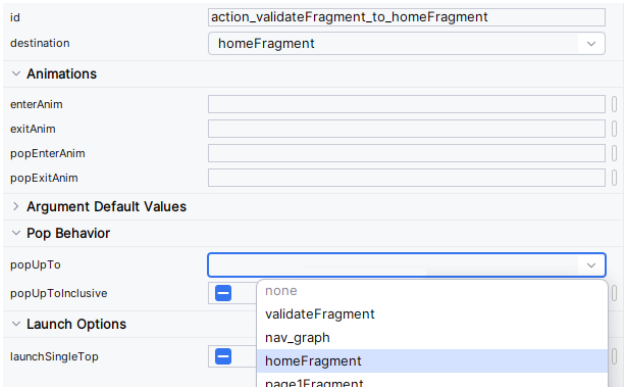
Add all screens/fragments in the graph and draw interactions between related screens.

Note the little house icon above the first fragment. It indicates the starting point of the app.



By default, screens will be added to the backstack, so you can press the back button. But in case you need, you can also set an action to pop the backstack to (and even including) a specified destination fragment.




## Step 4: code the interactions

Unfortunately, creating the graph and adding a nav host is not enough to make it work. We still have to code how all the interactions happen.

Navigating between two screens is done from the Kotlin code using a dedicated component called the nav controller. To retrieve your nav controller, fragments provide a *findNavController()* method which in turn provides a *navigate()* method you use to trigger an ***action*** by its id.

```
findNavController().navigate(R.id.action_mainFragment_to_page1Fragment)
```

*Step 5: pass parameters between fragments*

Nav controller *navigate()* function can also take our good old arguments Bundle object as a parameter to transmit data between fragments. But used along the navigation component, and acting as a wrapper around the Bundle mechanism, the SafeArgs module provides a nicer way to transmit parameters. Although including it can be a little convoluted.

1. Include safe-args plugin

First, add the global dependency to your top-level project *build.gradle* file

```
// Top-level build file where you can add configuration options common
// to all sub-projects/modules.

buildscript {
    repositories {
        google()
    }
    dependencies {
        val nav_version = "2.10.1"
        classpath("androidx.navigation:navigation-safe-args-gradle-plugin:$nav_version")
    }
}

plugins {
    . . .
}
```

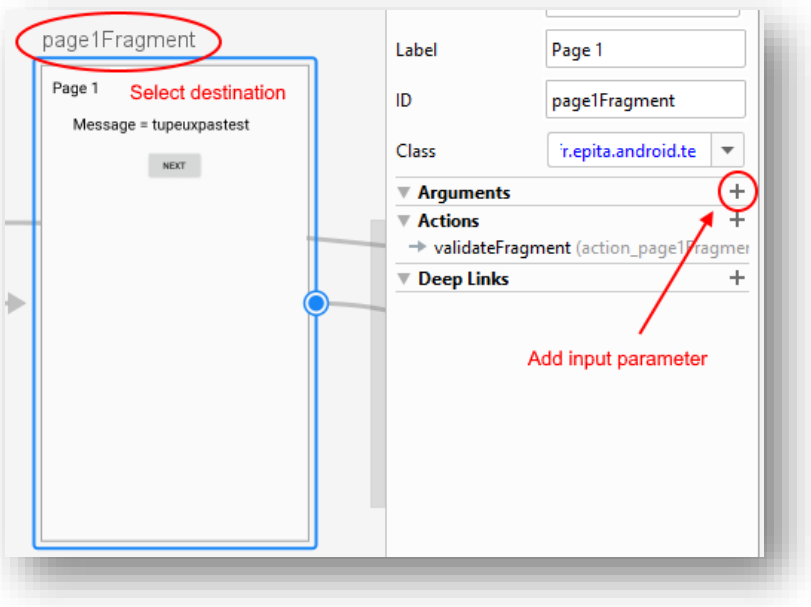
The apply safe-args plugin directly at module level *build.gradle*.

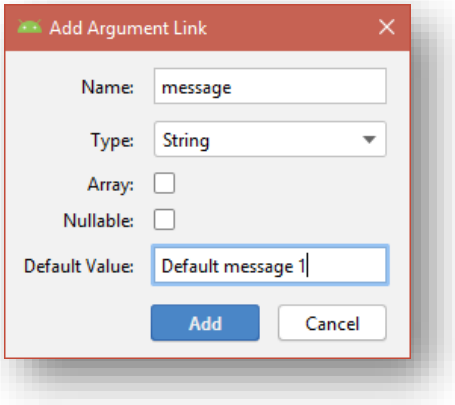
```
plugins {
    alias(libs.plugins.android.application)

    id("androidx.navigation.safeargs.kotlin")
}
```

### 2. Declare action parameters

Go back to your nav graph to declare each destination parameters.




### 3. Trigger actions using safe-args

The safe-args plugin creates two kinds of classes, the first of which is a wrapper around output actions for your fragments. Use this class to navigate from a fragment, instead of designating the action by its id.

**! ! ! Note:** before you can use generated classes, you might need to rebuild (menu Build -> Rebuild Project) your project.

```
findNavController().navigate(
        R.id.action_homeFragment_to_page1Fragment
)
    // now becomes...
findNavController().navigate(
        HomeFragmentDirections.actionHomeFragmentToPage1Fragment("Message content 1")
)
```

The other class type generated by safe-args is used on the receiving side of the action, i.e. the destination fragment. It can be retrieved using the ***navArgs()*** delegate.

```
private val args : Page1FragmentArgs by navArgs()
override fun onResume() {
    super.onResume()
    fragment_page1_txt_content.text = args.message
}
```

*Step 6: tie the actionBar*

Your app global and main look is set in the **manifest** (feel free to check it out) but this is just a pointer to your *theme.xml* file in *values* resource folder.

```
<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.HelloNavigation" parent="Theme.Material3.DayNight.NoActionBar">
    . . .
```

The quickest way to display an actionBar at the top of the screen is to set a base *theme* that has one. Several parameters can alter which ones are available in a particular project but recent values include

```
Theme.Material3.DayNight.NoActionBar / Theme.Material3.DayNight

Theme.AppCompat.Light.NoActionBar / Theme.AppCompat.Light.DarkActionBar
```

In our main activity, we can tie our nav graph to the actionbar, in which case we must also override the default “up” action button to pop our navigation backstack.

```
private lateinit var appBarConfiguration: AppBarConfiguration

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    . . .

    // extract nav controller from nav host because
    // findNavController() function not available yet in onCreate()
    // retrieve your nav host fragment
    val navHostFragment =
        supportFragmentManager
            .findFragmentById(R.id.activity_main_navhost) as NavHostFragment
    val navController: NavController = navHostFragment.navController

    // get a configuration object based upon your nav graph
    appBarConfiguration = AppBarConfiguration(navController.graph)
    // link the actionbar and the nav graph using this configuration
    setupActionBarWithNavController(navController, appBarConfiguration)
}


override fun onSupportNavigateUp(): Boolean {
    val navController = findNavController(R.id.activity_main_navhost)
    return navController.navigateUp(appBarConfiguration) ||
super.onSupportNavigateUp()
}
```

